NumPy, Matplotlib

Komputerowe wspomaganie decyzji

Przygotowanie środowiska: Podczas laboratoriów będziemy pracowali na ***notatnikach Jupyter*** *w języku Python. Pozwalają one na stopniową inkrementację kodu bez utraty poprzednich wyników, gdzie na każdym kroku możemy np. wizualizować nasze przekształcenia danych. Dostęp do potrzebnego nam środowiska mamy na Google Colab (z kontem Google). Lokalnie możemy zainstalować Python i Jupyter np. za pomocą platformy dystrybucyjnej Anaconda.*

1 NumPy

Rozpocznijmy od krótkiego wprowadzenia do biblioteki NumPy (link do dokumentacji), będącej podstawą do szybkich obliczeń w języku Python. Udostępnia ona wielowymiarowe obiekty tablicowe `ndarray` oraz zestaw procedur do szybkiego przetwarzania tablic, w tym operacji matematycznych, logicznych, manipulacji kształtami, sortowania, wybierania oraz bardziej zaawansowanych procedur. NumPy jest domyślnie zainstalowane na Google Colab, lokalnie możemy ją zainstalować za pomocą managera pakietów pip: `pip install numpy`.

Biblioteka NumPy jest tak naprawdę napisana w języku C. Jej podstawowym obiektem są tablice wielowymiarowe, zawierające elementy jednego typu. Wobec tego, nie działają one tak jak listy w Pythonie, gdzie każdy element listy jest dowolnym obiektem, ponieważ zależy nam na czasie obliczeń. Otwórzmy teraz dwie przykładowe obiekty `ndarray`:

```
import numpy as np

a = np.array([2, 3, 4])
B = np.array([[1.2, 2, 4], [3.1, 4.0, 5]])
```

Są one tworzone w tym wypadku na podstawie list. Ich typ `ndarray.dtype` będzie zadany poprzez podane elementy, np. dla typu `int` (Python) będzie to `int64` co odpowiada `long` w C (Linux). Jak widać nasze obiekty `ndarray` mogą reprezentować wektory, macierze, oraz tensory w przypadku większej ilości wymiarów. Podstawowe atrybuty tych obiektów to:

- `ndarray.ndim` - ilość wymiarów (ang. *axes*) tablicy.
- `ndarray.shape` - kształt tablicy, reprezentowany jako tuple
- `ndarray.size` - całkowita ilość elementów
- `ndarray.dtype` - typ elementów
- `ndarray.itemsize` - wielkość elementu w bajtach

Wypisz te atrybuty dla `a` oraz `b` i sprawdź czy zgadzają się one z twoimi przewidywaniami. Przypominając podstawy Pythona, każdy obiekt posiada pewne atrybuty (czyli metody, pola, wszystko "po kropce"), możemy je wypisać np. za pomocą:

```
dir(a)
```

Są one szczegółowo opisane tutaj. Przejdziemy teraz przez podstawy manipulowania obiektami `ndarray`.

1.1 Czas operacji

Spróbujmy teraz porównać czas operacji na listach w Pythonie, z czasem operacji na `ndarray`. Utwórzmy te dwa obiekty o długości `n`:

```
n = 1_000_000

lst = list(range(n)) # [0,1,2,3,...,n-1]
arr = np.arange(n)   # [0;1;2;....;n-1] (wektor pionowy)
```

Zmierzyć czas operacji możemy za pomocą `time.time()` (podającej obecny czas w sekundach), ale lepszym rozwiązaniem będzie skorzystanie z modułu timeit:

```
np_times = timeit.repeat(
    "arr.sum()",        # wykonywana funkcja
    globals=globals(),  # przekazanie kontekstu
    number=100,
    repeat=5,
)
```

Wykonujemy tutaj metodę `arr.sum()` 100 razy, co jest dodatkowo powtarzane pięć razy. Otrzymamy w wyniku listę czasów, gdzie każdy z czasów będzie odpowiadał 100 wykonaniom metody. Możemy teraz obliczyć średni czas operacji za pomocą funkcji `sum(np_times)` oraz następnego podzielenia otrzymanej wartości. Porównaj otrzymany czas dla numpy z:

- czasem wykonania wbudowanej funkcji sum dla listy `lst`,
- czasem wykonania samodzielnie zdefiniowanej funkcji obliczającej sumę listy `lst`.

1.2 Tworzenie tablic

Wcześniej zobaczyliśmy jak można utworzyć tablice `ndarray` na podstawie listy. NumPy oferuje także kilka przydanych funkcji aby uprościć ten proces:

`np.zeroes` - tworzy tablicę wypełnioną zerami,

`np.ones` - tworzy tablicę wypełnioną jedynkami,

`np.empty` - nie zainicjalizowana tablica,

Każda z tych funkcji przyjmuje jako argument kształt `ndarray.shape` docelowej tablicy:

```
C = np.ones((3,2))
```

Domyślnie typ danych utworzonej tablicy to `float64`, ale można go określić za pomocą argumentu `dtype`. Kolejną przydatną funkcją jest `np.linspace`, która pozwala na równomierne rozłożenie wartości w danym przedziale:

```
d = np.linspace(start=0,stop=1,num=6)
```

1.3 Podstawowe operacje

Operacje arytmetyczne są wykonywane dla każdego elementu osobno, przykładowo:

```
E = B - 1
F = E * 2
```

Jak wygląda obiekt `F` w porównaniu z `B.tolist()*2`?

Obiekty `ndarray` posiadają też pewne pomocne operacje jako metody:

```
B.min()             # najmniejsza wartość
B.max()             # największa
B.sum()             # suma elementów
B.sum(axis=0)       # suma dla każdej kolumny
B.sum(axis=1)       # suma dla każdego wiersza
```

Są zdefiniowane dwie podstawowe operacje mnożenia macierzy w NumPy:

- `*` - gwiazdka to iloczyn Hadamarda (mnożenie macierzy elementami, "naiwne")
- `@` - małpa to typowe mnożenie macierzy (Cauchy'ego)

1.3.1 Zadanie do wykonania

Pomnóż ze sobą macierze `E`, `B` oraz `E`, `C`. Wybierz odpowiedni sposób mnożenia dla tych przypadków na podstawie kształtu tych macierzy - Hadamarda albo Cauchy'ego.

1.4 Kształt tablicy

Zacznijmy od tego, że tablice można indeksować, przecinać (*slice*) i iterować podobnie jak typowe listy:

```
print(d[2])     # element o indeksie 2
print(d[-1])    # ostatni element
print(B[1][2])  # wiersz o indeksie 1 (drugi), kolumna o indeksie 2 (trzecia)
print(d[2:5])   # od elem. 2 (razem z 2) do 5 (bez 5)
print(d[2:])    # od 2 (razem z 2)
print(d[:2])    # pierwsze dwa elementy

for i in d:
    print(i)
```

Indeksując za pomocą liczb, podajemy które elementy wybrać. Ale możemy także indeksować za pomocą wartości logicznych `bool`, podając które elementy chcemy zachować jako `True`, a które elementy usunąć za pomocą `False`. Aby to zrobić potrzebujemy `ndarray` o tym samym kształcie jak `ndarray` z którego chcemy coś wybrać, co najprościej można zrobić to za pomocą maski:

```
B > 2
```

Z której możemy skorzystać według następującego schematu:

```
B[B > 2]        # wybranie elementów spełniających warunek
B[B > 2] = 0    # ustalenie wartości wybranych elementów
```

Możemy także utworzyć nową tablicę o zmienionym kształcie:

```
B.flatten()             # "spłaszczenie" do wektora
B.reshape(6,1)          # zmiana kształtu na podany
B.T                     # transpozycja
np.hsplit(B,3)          # podział poziomy
np.vstack((a, a-1))     # złożenie pionowe
np.hstack((a, a*-1))    # złożenie poziome
```

NumPy pozwala nam też na obliczenie odwrotności oraz wyznacznika macierzy:

```
from numpy.linalg import inv, det

inv(B@B.T)
det(B.T@B)
```

Przy czym należy pamiętać, że muszą to być macierze kwadratowe.

1.4.1 Zadania do wykonania

1. Utwórz na podstawie tablicy `d` taką macierz (skorzystaj z `reshape()`), aby możliwe było jej pomnożenie (Cauchy'ego) z macierzą `B`.
2. Oblicz sumę parzystych liczb zawartych w tablicy `t = np.arange(2,8)` (bez jawnej iteracji).
3. Wygenerujmy macierz zawierającą pomiary temperatury (w Kelwinach) przez 7 dni tygodnia:

```
temps = np.random.normal(
    loc=np.linspace(278, 303, 7)[:, None],
    scale=8,
    size=(7, 4)
)
temps[[1, 3], [2, 3]] = 0
```

Każdego dnia wykonano 4 pomiary temperatury, ale jak możesz zauważyć, dwukrotnie wystąpił błąd. Wykonaj teraz następujące operacje:

- Spróbuj zachować jedynie wartości większe od $0K$. Jak zmieni się `.shape` czyli kształt tablicy? Nie nadpisuj tutaj obiektu `temps` (pozostaw jej oryginalną postać do następnych poleceń).
- Zamień błędne pomiary temperatury ($0K$) na średnią temperaturę danego dnia - pamiętaj jednak, aby nie brać błędnych wartości pod uwagę przy liczeniu średniej. Możesz tutaj iterować przez kolejne dni.
- Zmień jednostkę z Kelwinów na stopnie Celsjusza ($0^oC = 273.15K$).
- Ile pomiarów temperatury było większe od 25^oC? Ile mniejsze od 0^oC?

2 Matplotlib

Biblioteka, którą będziemy stosowali do rysowania wykresów oraz wizualizacji danych to Matplotlib. Aby narysować wykres potrzebujemy najpierw zdefiniować dane w postaci list, lub innych indeksowanych typów danych (np. tablic NumPy), po czym korzystamy np. z funkcji `plt.scatter`:

```
import matplotlib.pyplot as plt
x = [0, 1, 2, 4]
y = [0, 1, 4, 16]

plt.scatter(x, y)
plt.xlabel('x')
plt.ylabel('$y=x^2$') # LaTeX

plt.show()
```

Aby wyrysować wykres funkcji w większej dokładności możemy posłużyć się wcześniej poznaną funkcją `np.linspace`:

```
x = np.linspace(0,4,100)
y = x**2

plt.plot(x,y)
plt.show()
```

Z bardziej szczegółowym formatowaniem wykresów zapoznamy się wykonując następne zadania. Dokumentacja Matplotlib jest dostępna tutaj.

2.1 Zadania do wykonania

Powróćmy do naszej tablicy NumPy zawierającej pomiary temperatury danego dnia (już z uzupełnionymi brakującymi temperaturami, oraz w stopniach Celsjusza). Przedstaw średnią temperaturę danego dnia na wykresie.

1. Dodaj tytuł wykresu za pomocą `plt.title("...")`.
2. Dodaj nazwy osi wykresu.
3. Dla osi y:
   - Ustal zakres (ylim) wykresu.
   - Ustal znaczniki osi (yticks), tak aby wykres był łatwy do odczytu. Możesz tutaj zdefiniować osobno grube (`major=True`) oraz drobne (`major=False`) znaczniki.
   - Dodaj odpowiednią podziałkę (grid) dla grubych oraz drobnych znaczników. Np. dla drobnych `plt.grid(which="minor", axis="y", linewidth=0.1)` doda lekkie linie poziome.
4. Dla osi x podobnie ustal zakres i znaczniki, przy czym wyświetl tutaj dni tygodnia. Czyli np. `plt.xticks(range(7), days)`, gdzie obiekt `days` to lista zawierająca nazwy dni tygodnia.
5. Zaznacz niebieską linią poziomą (axline) temperaturę 0^oC.
6. Zaznacz na wykresie maksymalną temperaturę w postaci adnotacji:
   ```
   plt.annotate(
       f"Max: {max_temp}°C",
       xy=(max_idx, max_temp),
       xytext=(-20, 10),
       textcoords="offset points"
   )
   ```
   Gdzie `max_idx`, `max_temp` to indeks oraz wartość maksymalnej temperatury. Możesz tutaj dobrać odpowiednio pozycję napisu za pomocą `xytext`.
7. Zapisz wykres do pliku w rozsądnej rozdzielczości, np. wywołując:
   `plt.savefig("temperature.png", dpi=300)` przed `plt.show()`.
   Istnieje także możliwość zapisania do pliku .pdf bez rasteryzacji obrazu.